

Questões

1. (Imperial College)¹ Crie uma classe `Robot` em que cada robô é identificado pelo seu nome e possui dois métodos: `introduce` e `remove`. A classe deve manter o controle do número total de robôs ativos (ou seja, robôs que foram criados) e deve ter um método que imprime o número de robôs ativos. O método `introduce` deve imprimir o nome do robô. O método `remove` deve diminuir o número de robôs ativos e imprimir uma mensagem apropriada, dependendo se ainda há robôs ativos ou não.

Copie e cole o exemplo abaixo para executar o seu código:

```
# Exemplo
robot1 = Robot("Robo1")
robot2 = Robot("Robo2")

robot1.introduce()
robot2.introduce()

robot1.print_active_robots()
# Robot.print_active_robots()

robot1.remove()
robot2.remove()

robot1.print_active_robots()
# Robot.print_active_robots()
```

2. (Downey)²

Escreva uma classe chamada `Circle` com os atributos `center` e `radius`, onde `center` é um objeto do tipo `Point` e `radius` é um número.

Instancie um objeto `Circle` que represente um círculo com seu centro em (150, 100) e raio 75.

Escreva uma função chamada `point_in_circle` que receba um `Circle` e um `Point` e retorne `True` se o `Point` estiver dentro ou na borda do círculo.

¹ Tradução livre

² Tradução livre

Escreva uma função chamada `rect_in_circle` que receba um `Circle` e um `Rectangle` e retorne `True` se o `Rectangle` estiver totalmente dentro ou na borda do círculo.

Escreva uma função chamada `rect_circle_overlap` que receba um `Circle` e um `Rectangle` e retorne `True` se qualquer um dos cantos do `Rectangle` estiver dentro do círculo.

Ou, para uma versão mais desafiadora, retorne `True` se qualquer parte do `Rectangle` estiver dentro do círculo.

3. Crie um programa em que os animais se movimentam em uma determinada área de floresta.

Crie uma classe `Animal`, que tenha como atributos `name` e `species`.

Além dos atributos anteriores, os animais devem ter um atributo `position`, que represente sua posição na floresta. A posição pode ser uma classe e contendo as coordenadas `x` e `y`.

A classe `Animal` deve permitir que uma nova posição seja atribuída diretamente ao animal, isto é, o atributo `position` deve ser público.

Crie uma classe `Forest`, que deve:

- Manter o controle de todos os animais presentes.
- Ter um método para adicionar um novo animal.
- Ter um método para remover um animal.
- Ter um método que imprime o número total de animais ativos na floresta.
- Ter um método que permite que todos os animais se movam simultaneamente, por valores específicos de coordenadas `x` e `y`, que devem ser passados como argumentos para o método.

Copie e cole o exemplo abaixo para executar código:

```
# Exemplo
forest = Forest()

animal1 = Animal("Aslan", "Leao", Position(10, 10))
animal2 = Animal("Puff", "Urso", Position(20, 20))

forest.add_animal(animal1)
forest.add_animal(animal2)
```

```
forest.print_total_animals()

forest.move_all_animals(5, 5)

animal1.position.x = 150
animal1.position.y = 150

print(f"{animal1.name} foi movido para a posição ({animal1.position.x},
{animal1.position.y}).")
```

4. Copie o código do Exercício 3 e modifique a classe `Forest`, introduzindo nela um tamanho específico. Por simplificação, considere que ela seja um retângulo e que um dos cantos tenha posição $x=0$ e $y=0$. Com isso, basta que a floresta tenha dois atributos extras, `width` e `height`, para que a posição de sua área esteja definida.

Em seguida, altere o programa para que valide se a nova posição atribuída ao animal é possível dentro dos limites da floresta. Para isso altere a classe `Position` para incluir *properties* e coloque todos os atributos dela como internos. Com isso, indicamos que não queremos que a posição do animal seja setada diretamente, mas validada antes pelo *property setter*.

O exemplo de uso código do exercício anterior não deve ser alterado (com exceção do construtor de `Forest`). Note como *properties* incluiu a validação sem quebrar a interface.